

Angular Route Protection - Complete Project Guide

Topic: Route Guards and Security Implementation

Objective: Build a secure Angular application with progressive route protection mechanisms

Introduction to Route Protection

In modern web applications, security is paramount. Not all pages should be accessible to all users. Consider these scenarios:

- A dashboard should only be accessible to logged-in users
- An admin panel should only be accessible to administrators
- Users should be warned before losing unsaved changes
- Data should be loaded before displaying a page

Angular provides powerful mechanisms called **Route Guards** to handle these requirements. In this comprehensive guide, we will build a complete application called the "**Secure Dashboard App**" that demonstrates all aspects of route protection.

Note:

This project follows a progressive approach, starting with basic routing and gradually adding security layers. Each step builds upon the previous one, ensuring a thorough understanding of how route protection works in Angular applications.

Project Overview: Secure Dashboard App

Our final application will include the following features:

- **Login System:** User authentication with different roles
- **Protected Dashboard:** Accessible only to authenticated users

- **Admin Section:** Restricted to users with admin privileges
- **Unsaved Form Warning:** Alerts users before leaving pages with unsaved changes
- **Data Pre-loading:** Ensures data is loaded before component initialization
- **Lazy-loaded Modules:** Optimized loading based on user permissions

 Important:

This is a learning project. In production applications, authentication should be handled server-side with proper security measures such as JWT tokens, HTTPS, and secure storage mechanisms.

STEP 0: Project Initialization

Before we begin implementing features, we need to create a new Angular project with routing enabled.

Creating the Angular Project

Open your terminal and execute the following commands:

```
ng new secure-app  
cd secure-app  
ng serve
```

Output:

Angular CLI Prompts:

? Would you like to add Angular routing? **Yes**

? Which stylesheet format would you like to use? **CSS**

✓ Packages installed successfully.

**** Angular Live Development Server is listening on localhost:4200 ****

Note:

What just happened? The Angular CLI created a new project with routing configuration. The `ng serve` command compiles the application and starts a development server. You can access the application by navigating to `http://localhost:4200` in your browser.

Understanding the Initial Structure

When routing is enabled, Angular CLI automatically generates:

- `app-routing.module.ts` - Defines application routes
- `<router-outlet>` in `app.component.html` - Placeholder for routed components

STEP 1: Basic Routing Structure

In this step, we'll create the fundamental pages of our application and set up basic routing without any security measures. This establishes the foundation upon which we'll layer protection mechanisms.

Generating Components

Angular CLI provides a convenient way to generate components. Execute these commands in your terminal:

```
ng g c pages/home
ng g c pages/login
ng g c pages/dashboard
```

Output:

```
CREATE src/app/pages/home/home.component.css (0 bytes)

CREATE src/app/pages/home/home.component.html (19 bytes)

CREATE src/app/pages/home/home.component.spec.ts (612 bytes)

CREATE src/app/pages/home/home.component.ts (196 bytes)

UPDATE src/app/app.module.ts (467 bytes)
```

(Similar output for login and dashboard components)

Note:

The `ng g c` command (short for `ng generate component`) creates four files for each component: HTML template, CSS styles, TypeScript class, and test specification. It also automatically registers the component in the application module.

Configuring Routes

Open `app-routing.module.ts` and define the routes for your application:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../pages/home/home.component';
import { LoginComponent } from '../pages/login/login.component';
import { DashboardComponent } from '../pages/dashboard/dashboard.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  { path: 'dashboard', component: DashboardComponent },
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
```

```
exports: [RouterModule]
})
export class AppRoutingModule { }
```

Note:

Route Configuration Explained:

- `path: ''` - Empty path represents the default/home route
- `path: 'login'` - Accessible via `http://localhost:4200/login`
- `component` - Specifies which component to display for each route

Creating Navigation

Update `app.component.html` to provide navigation links:

```
<nav style="padding: 20px; background-color: #f0f0f0; margin-bottom: 20px;">
  <a routerLink="/" style="margin-right: 15px;">Home</a> |
  <a routerLink="/login" style="margin: 0 15px;">Login</a> |
  <a routerLink="/dashboard" style="margin-left: 15px;">Dashboard</a>
</nav>

<div style="padding: 20px;">
  <router-outlet></router-outlet>
</div>
```

Output:

Visual Result: A navigation bar appears at the top with three links. Clicking each link changes the displayed component below without refreshing the entire page. This is Single Page Application (SPA) behavior in action.

Adding Content to Components

Update each component's HTML file to display meaningful content:

home.component.html

```
<div style="text-align: center; padding: 50px;">
  <h2>Welcome to Secure Dashboard App</h2>
  <p>This is the home page accessible to everyone.</p>
</div>
```

login.component.html

```
<div style="max-width: 400px; margin: 50px auto; padding: 30px; border: 1px solid #ddd;">
  <h2>Login</h2>
  <p>Please enter your credentials</p>
</div>
```

dashboard.component.html

```
<div style="padding: 30px;">
  <h2>Dashboard</h2>
  <p>This is the main dashboard area.</p>
</div>
```

✓ Success:

Achievement Unlocked: Basic routing is now functional. Users can navigate between pages freely.

✗ Issue:

Current Problem: The dashboard is accessible to everyone, even users who haven't logged in. This is a security vulnerability that we'll address in the next steps.

STEP 2: Authentication Service

Before implementing route protection, we need a mechanism to manage user authentication state. An **Authentication Service** centralizes all authentication-related logic.

Understanding Services in Angular

Services are singleton objects that provide specific functionality across the application. They are ideal for:

- Sharing data between components
- Handling business logic
- Making HTTP requests
- Managing application state

Generating the Auth Service

```
ng g s services/auth
```

Output:

```
CREATE src/app/services/auth.service.spec.ts (347 bytes)
```

```
CREATE src/app/services/auth.service.ts (133 bytes)
```

Implementing Authentication Logic

Open `auth.service.ts` and implement the following code:

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class AuthService {
```

```
private loggedIn = false;
private role = '';

constructor() { }

/**
 * Authenticates a user based on username and password
 * In production, this would make an HTTP request to a server
 */
login(username: string, password: string): boolean {
  // Admin user authentication
  if (username === 'admin' && password === 'admin123') {
    this.loggedIn = true;
    this.role = 'admin';
    return true;
  }

  // Regular user authentication
  if (username === 'user' && password === 'user123') {
    this.loggedIn = true;
    this.role = 'user';
    return true;
  }

  // Authentication failed
  return false;
}

/**
 * Logs out the current user
 */
logout(): void {
  this.loggedIn = false;
  this.role = '';
}

/**
 * Checks if a user is currently logged in
 */
isLoggedIn(): boolean {
  return this.loggedIn;
}

/**
 * Returns the role of the current user
 */
```



```
getRole(): string {  
    return this.role;  
}  
}
```

Note:

Service Architecture Explained:

- `@Injectable({ providedIn: 'root' })` - Makes service available application-wide
- `private loggedIn` - Stores authentication state
- `private role` - Stores user role for authorization
- `login()` - Validates credentials and updates state
- `isLoggedIn()` - Used by guards to check authentication status

Implementing Login Component

Update `login.component.ts` to use the authentication service:

```
import { Component } from '@angular/core';  
import { Router } from '@angular/router';  
import { AuthService } from '../../services/auth.service';  
  
@Component({  
    selector: 'app-login',  
    templateUrl: './login.component.html',  
    styleUrls: ['./login.component.css']  
})  
export class LoginComponent {  
  
    username = '';  
    password = '';  
    errorMessage = '';  
  
    constructor(  
        private auth: AuthService,  
        private router: Router  
    ) {}  
}
```

```
login(): void {
  if (this.auth.login(this.username, this.password)) {
    // Login successful - navigate to dashboard
    this.router.navigate(['/dashboard']);
  } else {
    // Login failed - show error
    this.errorMessage = 'Invalid username or password';
  }
}
}
```

Creating Login Form

Update `login.component.html` :

```
<div style="max-width: 400px; margin: 50px auto; padding: 30px; border: 1px solid #ddd; border-
radius: 8px;">
  <h2 style="text-align: center;">Login</h2>

  <div style="margin-bottom: 15px;">
    <label style="display: block; margin-bottom: 5px;">Username:</label>
    <input
      type="text"
      [(ngModel)]="username"
      style="width: 100%; padding: 8px; box-sizing: border-box;"
      placeholder="Enter username">
    </div>

    <div style="margin-bottom: 15px;">
      <label style="display: block; margin-bottom: 5px;">Password:</label>
      <input
        type="password"
        [(ngModel)]="password"
        style="width: 100%; padding: 8px; box-sizing: border-box;"
        placeholder="Enter password">
      </div>

    <button
      (click)="login()"
      style="width: 100%; padding: 10px; background-color: #007bff; color: white; border: none;
border-radius: 4px; cursor: pointer;">
      Login
    </button>
```

```
<div *ngIf="errorMessage" style="margin-top: 15px; padding: 10px; background-color: #f8d7da; color: #721c24; border-radius: 4px;">
  {{ errorMessage }}
</div>

<div style="margin-top: 20px; padding: 15px; background-color: #e7f3ff; border-radius: 4px;">
  <p style="margin: 0; font-size: 0.9em;"><strong>Test Credentials:</strong></p>
  <p style="margin: 5px 0; font-size: 0.9em;">Admin: admin / admin123</p>
  <p style="margin: 5px 0; font-size: 0.9em;">User: user / user123</p>
</div>
</div>
```

⚠ Important:

To use `[(ngModel)]`, you must import `FormsModule` in `app.module.ts`:

```
import { FormsModule } from '@angular/forms';

@NgModule({
  imports: [
    // ... other imports
    FormsModule
  ]
})
```

Output:

Visual Result: A login form appears with username and password fields. Users can enter credentials and click the Login button. Successful authentication redirects to the dashboard, while failed attempts display an error message.

✅ Success:

Achievement Unlocked: Login functionality is operational.

✖ Issue:

Current Problem: While login works, users can still bypass it by directly typing `/dashboard` in the browser address bar. We need route guards to prevent this.

STEP 3: CanActivate Guard - Basic Route Protection

The `CanActivate` guard is the most fundamental route protection mechanism in Angular. It determines whether a route can be activated based on custom logic.

Understanding CanActivate

The `CanActivate` interface allows you to:

- Check user authentication status before route activation
- Redirect unauthorized users to login page
- Implement custom authorization logic
- Return boolean, Promise, or Observable values

Generating the Auth Guard

```
ng g guard guards/auth
```

Output:

? Which interfaces would you like to implement? **CanActivate**

CREATE src/app/guards/auth.guard.spec.ts (331 bytes)

CREATE src/app/guards/auth.guard.ts (456 bytes)

Implementing the Guard Logic

Open `auth.guard.ts` and implement:

```
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {

  constructor(
    private auth: AuthService,
    private router: Router
  ) {}

  canActivate(): boolean {
    // Check if user is authenticated
    if (this.auth.isLoggedIn()) {
      // User is authenticated - allow access
      return true;
    }

    // User is not authenticated - redirect to login
    console.log('Access denied. Redirecting to login...');
    this.router.navigate(['/login']);
    return false;
  }
}
```

Note:

How CanActivate Works:

1. User attempts to navigate to a protected route
2. Angular calls the `canActivate()` method
3. Method checks authentication status via `AuthService`
4. If authenticated: returns `true` , navigation proceeds

5. If not authenticated: returns `false` and redirects to login

Applying the Guard to Routes

Update `app-routing.module.ts` to protect the dashboard:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { HomeComponent } from '../pages/home/home.component';
import { LoginComponent } from '../pages/login/login.component';
import { DashboardComponent } from '../pages/dashboard/dashboard.component';
import { AuthGuard } from '../guards/auth.guard';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard] // ← Guard applied here
  },
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

Output:

Testing the Guard:

1. Navigate to `http://localhost:4200/dashboard` without logging in
2. Result: Automatically redirected to `/login`
3. Log in with valid credentials
4. Navigate to `/dashboard`

5. Result: Dashboard displays successfully

✓ Success:

Achievement Unlocked: Dashboard is now protected. Unauthenticated users cannot access it, even by typing the URL directly.

Enhancing User Experience

Let's add a logout button and show the current authentication status. Update `app.component.ts` :

```
import { Component } from '@angular/core';
import { Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
})
export class AppComponent {

  constructor(
    public auth: AuthService,
    private router: Router
  ) {}

  logout(): void {
    this.auth.logout();
    this.router.navigate(['/']);
  }
}
```

Update `app.component.html` :

```
<nav style="padding: 20px; background-color: #f0f0f0; margin-bottom: 20px; display: flex;
justify-content: space-between; align-items: center;">
  <div>
```

```

    <a routerLink="/" style="margin-right: 15px;">Home</a> |
    <a routerLink="/login" style="margin: 0 15px;">Login</a> |
    <a routerLink="/dashboard" style="margin-left: 15px;">Dashboard</a>
  </div>

  <div *ngIf="auth.isLoggedIn()" style="display: flex; align-items: center; gap: 15px;">
    <span style="color: #28a745;">● Logged in as: <strong>{{ auth.getRole() }}</strong></span>
    <button
      (click)="logout()"
      style="padding: 5px 15px; background-color: #dc3545; color: white; border: none; border-
radius: 4px; cursor: pointer;">
      Logout
    </button>
  </div>
</nav>

<div style="padding: 20px;">
  <router-outlet></router-outlet>
</div>

```

Output:

Enhanced Interface: After login, the navigation bar displays the current user role (admin/user) and a logout button. Clicking logout clears the session and redirects to home.

STEP 4: CanDeactivate Guard - Unsaved Changes Protection

The `CanDeactivate` guard prevents users from accidentally leaving a page with unsaved changes. This is crucial for forms and editors where data loss could be problematic.

Understanding CanDeactivate

Unlike `CanActivate` which guards route entry, `CanDeactivate` guards route exit. It's commonly used for:

- Confirming before discarding form changes
- Saving draft content automatically
- Preventing accidental navigation away from unsaved work

Creating the Edit Profile Component

```
ng g c pages/edit-profile
```

Defining the CanLeave Interface

Create a new file `can-leave.interface.ts` in the `guards` folder:

```
export interface CanLeave {  
  canDeactivate(): boolean;  
}
```

Note:

This interface ensures that any component protected by the deactivate guard implements a `canDeactivate()` method, establishing a contract between the guard and the component.

Generating the Deactivate Guard

```
ng g guard guards/deactivate
```

Output:

? Which interfaces would you like to implement? **CanDeactivate**

CREATE src/app/guards/deactivate.guard.ts

Implementing the Guard

Open `deactivate.guard.ts` :

```
import { Injectable } from '@angular/core';
import { CanDeactivate } from '@angular/router';
import { CanLeave } from './can-leave.interface';

@Injectable({
  providedIn: 'root'
})
export class DeactivateGuard implements CanDeactivate<CanLeave> {

  canDeactivate(component: CanLeave): boolean {
    // Delegate the decision to the component itself
    return component.canDeactivate();
  }
}
```

 Note:

Design Pattern: The guard delegates the decision to the component, allowing each component to implement its own logic for determining whether navigation away should be allowed.

Implementing the Edit Profile Component

Update `edit-profile.component.ts`:

```
import { Component } from '@angular/core';
import { CanLeave } from '../guards/can-leave.interface';

@Component({
  selector: 'app-edit-profile',
  templateUrl: './edit-profile.component.html',
  styleUrls: ['./edit-profile.component.css']
})
export class EditProfileComponent implements CanLeave {

  // Tracks whether form has unsaved changes
  dirty = false;

  // User data
  userName = '';
  userEmail = '';
```

```

/**
 * Called when any form field changes
 */
onFieldChange(): void {
  this.dirty = true;
}

/**
 * Simulates saving data
 */
save(): void {
  console.log('Saving profile...');
  // In real application, make HTTP request here
  this.dirty = false;
  alert('Profile saved successfully!');
}

/**
 * Called by the CanDeactivate guard
 * Returns true if navigation is allowed, false otherwise
 */
canDeactivate(): boolean {
  if (this.dirty) {
    // Show confirmation dialog
    return confirm('You have unsaved changes. Do you really want to leave?');
  }
  // No unsaved changes - allow navigation
  return true;
}
}

```

Creating the Edit Form

Update `edit-profile.component.html` :

```

<div style="max-width: 600px; margin: 30px auto; padding: 30px; border: 1px solid #ddd; border-
radius: 8px;">
  <h2>Edit Profile</h2>

  <div style="margin-bottom: 20px;">
    <label style="display: block; margin-bottom: 5px; font-weight: 600;">Name:</label>
    <input
      type="text"

```

```

    [(ngModel)]="userName"
    (input)="onFieldChange()"
    style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;"
    placeholder="Enter your name">
  </div>

  <div style="margin-bottom: 20px;">
    <label style="display: block; margin-bottom: 5px; font-weight: 600;">Email:</label>
    <input
      type="email"
      [(ngModel)]="userEmail"
      (input)="onFieldChange()"
      style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;"
      placeholder="Enter your email">
    </div>

    <button
      (click)="save()"
      style="padding: 10px 30px; background-color: #28a745; color: white; border: none; border-
radius: 4px; cursor: pointer; margin-right: 10px;">
      Save Changes
    </button>

    <a
      routerLink="/"
      style="padding: 10px 30px; background-color: #6c757d; color: white; text-decoration: none;
border-radius: 4px; display: inline-block;">
      Cancel
    </a>

    <div *ngIf="dirty" style="margin-top: 20px; padding: 15px; background-color: #fff3cd; border-
left: 4px solid #ffc107; border-radius: 4px;">
      <strong><img alt="Warning icon" data-bbox="208 655 225 668"/> Unsaved Changes</strong>
      <p style="margin: 5px 0 0 0;">You have unsaved changes. Don't forget to save before
leaving!</p>
    </div>
  </div>

```

Configuring the Route with Guard

Update `app-routing.module.ts`:

```
import { DeactivateGuard } from './guards/deactivate.guard';
import { EditProfileComponent } from './pages/edit-profile/edit-profile.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'edit',
    component: EditProfileComponent,
    canActivate: [AuthGuard], // Must be logged in
    canDeactivate: [DeactivateGuard] // Check for unsaved changes
  }
];
```

Output:

Testing CanDeactivate:

1. Log in and navigate to `/edit`
2. Type something in the name or email field
3. Try to navigate away (click Home or Dashboard link)
4. Result: A browser confirmation dialog appears
5. Click "Cancel" - stays on page
6. Click "OK" - navigates away (changes lost)
7. Click "Save Changes" button - changes saved, dirty flag cleared
8. Now navigate away - no confirmation needed

☑ Success:

Achievement Unlocked: Users are now protected from accidentally losing unsaved form data.

STEP 5: Role-Based Access Control

Different users should have different levels of access. An admin should access areas that regular users cannot. Role-based guards implement this authorization layer.

Understanding Authorization vs Authentication

Authentication	Authorization
Who are you?	What can you do?
Verifies identity	Verifies permissions
Example: Login with username/password	Example: Admin-only features

Generating the Role Guard

```
ng g guard guards/role
```

Output:

? Which interfaces would you like to implement? **CanActivate**

Implementing Role-Based Protection

Open `role.guard.ts`:

```
import { Injectable } from '@angular/core';
import { CanActivate, ActivatedRouteSnapshot, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root'
})
export class RoleGuard implements CanActivate {
```

```
constructor(  
  private auth: AuthService,  
  private router: Router  
) {}  
  
canActivate(route: ActivatedRouteSnapshot): boolean {  
  
  // Get the required role from route data  
  const requiredRole = route.data['role'];  
  
  // Get the user's current role  
  const userRole = this.auth.getRole();  
  
  // Check if user has the required role  
  if (userRole === requiredRole) {  
    console.log(`Access granted: User has ${requiredRole} role`);  
    return true;  
  }  
  
  // Access denied - redirect to home  
  console.log(`Access denied: User is ${userRole}, but ${requiredRole} required`);  
  alert(`Access Denied: This area requires ${requiredRole} privileges`);  
  this.router.navigate(['/']);  
  return false;  
}  
}
```

Note:

ActivatedRouteSnapshot: This object contains information about the route being activated, including URL parameters, query parameters, and route data. We use `route.data['role']` to retrieve the required role specified in the route configuration.

Creating Admin Component

```
ng g c pages/admin
```

Update `admin.component.html` :

```
<div style="padding: 30px; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white; border-radius: 8px; margin: 20px;">
  <h2 style="margin-top: 0;"><img alt="lock icon" data-bbox="348 178 362 192"/> Admin Control Panel</h2>
  <p>Welcome to the administrative area. Only users with admin privileges can access this
section.</p>

  <div style="background-color: rgba(255,255,255,0.2); padding: 20px; border-radius: 4px;
margin-top: 20px;">
    <h3>Admin Features:</h3>
    <ul>
      <li>User Management</li>
      <li>System Configuration</li>
      <li>Reports and Analytics</li>
      <li>Security Settings</li>
    </ul>
  </div>
</div>
```

Configuring Protected Route

Update `app-routing.module.ts`:

```
import { RoleGuard } from './guards/role.guard';
import { AdminComponent } from './pages/admin/admin.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'edit',
    component: EditProfileComponent,
    canActivate: [AuthGuard],
    canDeactivate: [DeactivateGuard]
  },
  {
    path: 'admin',
    component: AdminComponent,
    canActivate: [AuthGuard, RoleGuard], // ← Multiple guards
```



```
data: { role: 'admin' } // ← Required role
}
];
```

Note:

Multiple Guards: When multiple guards are specified, they execute in order. The route is only activated if ALL guards return true. Here, `AuthGuard` first verifies login status, then `RoleGuard` verifies admin privileges.

Adding Admin Link to Navigation

Update `app.component.html` to conditionally show admin link:

```
<nav style="padding: 20px; background-color: #f0f0f0; margin-bottom: 20px; display: flex;
justify-content: space-between; align-items: center;">
  <div>
    <a routerLink="/" style="margin-right: 15px;">Home</a> |
    <a routerLink="/login" style="margin: 0 15px;">Login</a> |
    <a routerLink="/dashboard" style="margin: 0 15px;">Dashboard</a>
    <span *ngIf="auth.getRole() === 'admin'">
      | <a routerLink="/admin" style="margin-left: 15px; color: #764ba2; font-weight: 600;">🔒
Admin</a>
    </span>
  </div>

  <div *ngIf="auth.isLoggedIn()" style="display: flex; align-items: center; gap: 15px;">
    <span style="color: #28a745;">● Logged in as: <strong>{{ auth.getRole() }}</strong></span>
    <button
      (click)="logout()"
      style="padding: 5px 15px; background-color: #dc3545; color: white; border: none; border-
radius: 4px; cursor: pointer;">
      Logout
    </button>
  </div>
</nav>
```

Output:

Testing Role-Based Access:

1. Login as regular user (username: `user` , password: `user123`)
2. Try to access `/admin` by typing URL
3. Result: Access denied alert, redirected to home
4. Admin link is hidden in navigation
5. Logout and login as admin (username: `admin` , password: `admin123`)
6. Admin link appears in navigation
7. Click admin link or navigate to `/admin`
8. Result: Admin panel displays successfully

✓ Success:

Achievement Unlocked: Role-based authorization is functional. Different users see different parts of the application based on their privileges.

STEP 6: CanActivateChild - Protecting Child Routes

When you have a parent route with multiple child routes, protecting each child individually is repetitive. `CanActivateChild` allows you to protect all child routes with a single guard on the parent.

Understanding Route Hierarchy

Angular supports nested routing structures:

```
/admin          ← Parent route
├── /admin/users  ← Child route
├── /admin/settings ← Child route
└── /admin/reports ← Child route
```

Creating Child Components

```
ng g c admin/users
ng g c admin/settings
```

Implementing Child Component Content

users.component.html

```
<div style="padding: 20px; background-color: #e7f3ff; border-radius: 4px; margin: 10px 0;">
  <h3>👤 User Management</h3>
  <p>Manage system users, permissions, and access levels.</p>

  <table style="width: 100%; margin-top: 15px; background-color: white; border-radius: 4px;">
    <thead>
      <tr style="background-color: #f0f0f0;">
        <th style="padding: 10px; text-align: left;">User ID</th>
        <th style="padding: 10px; text-align: left;">Name</th>
        <th style="padding: 10px; text-align: left;">Role</th>
        <th style="padding: 10px; text-align: left;">Status</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td style="padding: 10px;">001</td>
        <td style="padding: 10px;">Admin User</td>
        <td style="padding: 10px;">Administrator</td>
        <td style="padding: 10px; color: #28a745;">● Active</td>
      </tr>
      <tr style="background-color: #f9f9f9;">
        <td style="padding: 10px;">002</td>
        <td style="padding: 10px;">Regular User</td>
        <td style="padding: 10px;">User</td>
        <td style="padding: 10px; color: #28a745;">● Active</td>
      </tr>
    </tbody>
  </table>
</div>
```

settings.component.html

```
<div style="padding: 20px; background-color: #fff3cd; border-radius: 4px; margin: 10px 0;">
  <h3>⚙️ System Settings</h3>
  <p>Configure system-wide preferences and parameters.</p>

  <div style="background-color: white; padding: 15px; border-radius: 4px; margin-top: 15px;">
    <h4>Application Settings</h4>
    <label style="display: block; margin: 10px 0;">
      <input type="checkbox" checked=""> Enable email notifications
    </label>
    <label style="display: block; margin: 10px 0;">
      <input type="checkbox" checked=""> Maintenance mode
    </label>
    <label style="display: block; margin: 10px 0;">
      <input type="checkbox" checked=""> Two-factor authentication
    </label>
  </div>
</div>
```

Generating CanActivateChild Guard

```
ng g guard guards/admin-child
```

Output:

? Which interfaces would you like to implement? **CanActivateChild**

Implementing the Child Guard

Open `admin-child.guard.ts` :

```
import { Injectable } from '@angular/core';
import { CanActivateChild, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root'
```

```

})
export class AdminChildGuard implements CanActivateChild {

  constructor(
    private auth: AuthService,
    private router: Router
  ) {}

  canActivateChild(): boolean {

    // Check if user has admin role
    if (this.auth.getRole() === 'admin') {
      console.log('Child route access granted: User is admin');
      return true;
    }

    // Not an admin - deny access
    console.log('Child route access denied: User is not admin');
    alert('Admin privileges required to access this section');
    this.router.navigate(['/']);
    return false;
  }
}

```

Note:

Efficiency Benefit: Instead of adding `canActivate: [RoleGuard]` to each child route, we add `canActivateChild: [AdminChildGuard]` once to the parent. All children are automatically protected.

Configuring Parent and Child Routes

Update `admin.component.html` to include child navigation:

```

<div style="padding: 30px; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white; border-radius: 8px; margin: 20px;">
  <h2 style="margin-top: 0;"><img alt="lock icon" data-bbox="348 762 362 776"/> Admin Control Panel</h2>
  <p>Welcome to the administrative area. Only users with admin privileges can access this
section.</p>

  <nav style="background-color: rgba(255,255,255,0.2); padding: 15px; border-radius: 4px;
margin: 20px 0;">

```

```

    <a routerLink="/admin/users" routerLinkActive="active" style="color: white; margin-right:
20px; text-decoration: none; padding: 5px 10px; background-color: rgba(255,255,255,0.1);
border-radius: 4px;">👤 Users</a>
    <a routerLink="/admin/settings" routerLinkActive="active" style="color: white; text-
decoration: none; padding: 5px 10px; background-color: rgba(255,255,255,0.1); border-radius:
4px;">⚙️ Settings</a>
  </nav>

  <!-- Child routes will render here -->
  <router-outlet></router-outlet>
</div>

```

Update `app-routing.module.ts` with child routes:

```

import { AdminChildGuard } from './guards/admin-child.guard';
import { UsersComponent } from './admin/users/users.component';
import { SettingsComponent } from './admin/settings/settings.component';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'edit',
    component: EditProfileComponent,
    canActivate: [AuthGuard],
    canDeactivate: [DeactivateGuard]
  },
  {
    path: 'admin',
    component: AdminComponent,
    canActivate: [AuthGuard, RoleGuard],
    canActivateChild: [AdminChildGuard], // ← Protects all children
    data: { role: 'admin' },
    children: [
      { path: 'users', component: UsersComponent },
      { path: 'settings', component: SettingsComponent }
    ]
  }
]

```

```
]
}
];
```

Output:

Testing Child Routes:

1. Login as admin
2. Navigate to `/admin`
3. Click "Users" link
4. Result: Users component displays within admin panel
5. Click "Settings" link
6. Result: Settings component displays within admin panel
7. Logout and login as regular user
8. Try to access `/admin/users` directly
9. Result: Both parent and child guards block access

✓ Success:

Achievement Unlocked: Nested routes are efficiently protected with a single guard on the parent route.

STEP 7: Resolver - Pre-loading Data

A common problem in web applications is displaying components before their required data is loaded, resulting in empty screens or loading spinners. **Resolvers** solve this by ensuring data is available before the component activates.

Understanding Resolvers

A Resolver is a service that:

- Fetches data BEFORE the route activates
- Prevents the component from displaying until data is ready
- Makes data available through the ActivatedRoute
- Can return Observables, Promises, or direct values

Creating User Service

```
ng g s services/user
```

Open `user.service.ts`:

```
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { delay } from 'rxjs/operators';

export interface User {
  id: number;
  name: string;
  email: string;
  role: string;
  joinedDate: string;
}

@Injectable({
  providedIn: 'root'
})
export class UserService {

  // Simulated database of users
  private users: User[] = [
    { id: 1, name: 'John Doe', email: 'john@example.com', role: 'Admin', joinedDate: '2023-01-15' },
    { id: 2, name: 'Jane Smith', email: 'jane@example.com', role: 'User', joinedDate: '2023-03-20' },
    { id: 3, name: 'Bob Johnson', email: 'bob@example.com', role: 'User', joinedDate: '2023-05-10' }
  ];
```



```
constructor() { }

/**
 * Simulates an HTTP request to fetch user by ID
 * The delay simulates network latency
 */
getUser(id: number): Observable<User | undefined> {
  const user = this.users.find(u => u.id === id);
  // Simulate 1 second delay
  return of(user).pipe(delay(1000));
}
```

Note:

RxJS Operators:

- `of()` - Creates an Observable from a value
- `delay(1000)` - Delays emission by 1000ms (simulating API call)
- `pipe()` - Chains operators together

Generating the Resolver

```
ng g resolver resolvers/user
```

Output:

```
CREATE src/app/resolvers/user.resolver.spec.ts
```

```
CREATE src/app/resolvers/user.resolver.ts
```

Implementing the Resolver

Open `user.resolver.ts` :

```
import { Injectable } from '@angular/core';
import { Resolve, ActivatedRouteSnapshot } from '@angular/router';
import { Observable } from 'rxjs';
import { UserService, User } from '../services/user.service';

@Injectable({
  providedIn: 'root'
})
export class UserResolver implements Resolve<User | undefined> {

  constructor(private userService: UserService) {}

  resolve(route: ActivatedRouteSnapshot): Observable<User | undefined> {

    // Extract user ID from route parameters
    const userId = +route.paramMap.get('id')!;

    console.log(`Resolver: Fetching user with ID ${userId}...`);

    // Fetch and return user data
    // Navigation will wait until this Observable completes
    return this.userService.getUser(userId);
  }
}
```

Note:

Key Points:

- `+route.paramMap.get('id')!` - The `+` converts string to number
- `!` (non-null assertion) tells TypeScript we're confident the value exists
- Angular waits for the Observable to emit before activating the component

Creating User Detail Component

```
ng g c pages/user-detail
```

Update `user-detail.component.ts` :

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { User } from '../../services/user.service';

@Component({
  selector: 'app-user-detail',
  templateUrl: './user-detail.component.html',
  styleUrls: ['./user-detail.component.css']
})
export class UserDetailsComponent implements OnInit {

  user: User | undefined;

  constructor(private route: ActivatedRoute) {}

  ngOnInit(): void {
    // Data is already loaded by the resolver
    // Access it from route snapshot
    this.user = this.route.snapshot.data['user'];

    console.log('Component initialized with pre-loaded user:', this.user);
  }
}
```

Update `user-detail.component.html` :

```
<div style="max-width: 600px; margin: 30px auto; padding: 30px; border: 1px solid #ddd; border-
radius: 8px;">

  <div *ngIf="user" style="background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white; padding: 30px; border-radius: 8px; margin-bottom: 20px;">
    <h2 style="margin-top: 0;">👤 User Profile</h2>
    <p style="opacity: 0.9;">Detailed information about the selected user</p>
  </div>

  <div *ngIf="user">
    <table style="width: 100%; border-collapse: collapse;">
      <tr style="border-bottom: 1px solid #ddd;">
        <td style="padding: 15px; font-weight: 600; width: 40%;">User ID:</td>
        <td style="padding: 15px;">{{ user.id }}</td>
      </tr>
      <tr style="border-bottom: 1px solid #ddd;">
        <td style="padding: 15px; font-weight: 600;">Name:</td>
        <td style="padding: 15px;">{{ user.name }}</td>
      </tr>
    </table>
  </div>
</div>
```

```

    </tr>
    <tr style="border-bottom: 1px solid #ddd;">
      <td style="padding: 15px; font-weight: 600;">Email:</td>
      <td style="padding: 15px;">{{ user.email }}</td>
    </tr>
    <tr style="border-bottom: 1px solid #ddd;">
      <td style="padding: 15px; font-weight: 600;">Role:</td>
      <td style="padding: 15px;">
        <span style="background-color: #28a745; color: white; padding: 3px 10px; border-
radius: 12px; font-size: 0.85em;">
          {{ user.role }}
        </span>
      </td>
    </tr>
    <tr>
      <td style="padding: 15px; font-weight: 600;">Joined Date:</td>
      <td style="padding: 15px;">{{ user.joinedDate }}</td>
    </tr>
  </table>

  <button
    routerLink="/dashboard"
    style="margin-top: 20px; padding: 10px 30px; background-color: #007bff; color: white;
border: none; border-radius: 4px; cursor: pointer;">
    ← Back to Dashboard
  </button>
</div>

<div *ngIf="!user" style="text-align: center; padding: 30px; color: #dc3545;">
  <h3>✖ User Not Found</h3>
  <p>The requested user does not exist.</p>
  <button
    routerLink="/dashboard"
    style="padding: 10px 30px; background-color: #6c757d; color: white; border: none; border-
radius: 4px; cursor: pointer;">
    Return to Dashboard
  </button>
</div>

</div>

```

Configuring Route with Resolver

Update `app-routing.module.ts` :

```
import { UserResolver } from '../resolvers/user.resolver';
import { UserDetailComponent } from '../pages/user-detail/user-detail.component';

const routes: Routes = [
  // ... existing routes ...
  {
    path: 'user/:id',
    component: UserDetailComponent,
    canActivate: [AuthGuard],
    resolve: { user: UserResolver } // ← Resolver configuration
  }
];
```

Note:

The key `'user'` in `resolve: { user: UserResolver }` determines how to access the data in the component: `route.snapshot.data['user']`

Adding User Links to Dashboard

Update `dashboard.component.html` to include user links:

```
<div style="padding: 30px;">
  <h2>&img alt="dashboard icon" data-bbox="158 592 178 605"/> Dashboard</h2>
  <p>Welcome to your personal dashboard. This area is protected and requires authentication.</p>

  <div style="margin-top: 30px; background-color: #f8f9fa; padding: 20px; border-radius: 8px;">
    <h3>User Directory</h3>
    <p>Click on a user to view their profile (demonstrates resolver functionality):</p>

    <ul style="list-style: none; padding: 0;">
      <li style="margin: 10px 0;">
        <a routerLink="/user/1" style="display: block; padding: 15px; background-color: white;
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
          &img alt="user icon" data-bbox="192 778 208 792"/> View John Doe's Profile
        </a>
      </li>
      <li style="margin: 10px 0;">
        <a routerLink="/user/2" style="display: block; padding: 15px; background-color: white;
```

```
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
   View Jane Smith's Profile
</a>
</li>
<li style="margin: 10px 0;">
  <a routerLink="/user/3" style="display: block; padding: 15px; background-color: white;
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
     View Bob Johnson's Profile
  </a>
</li>
</ul>
</div>
</div>
```

Output:

Testing the Resolver:

1. Login and navigate to dashboard
2. Click on "View John Doe's Profile"
3. Observe: There's a 1-second delay before the page loads
4. Result: User profile displays with all data immediately available
5. Open browser console to see resolver logs
6. No "loading..." spinners needed - data is guaranteed to be ready

☑ Success:

Achievement Unlocked: Data is pre-loaded before component initialization, ensuring a smooth user experience without flickering or empty states.

⚠ Important:

Resolver Considerations: While resolvers ensure data availability, they delay navigation. For slow API calls, consider showing a route-level loading indicator or using loading skeletons within components instead.

STEP 8: Lazy Loading with CanMatch

As applications grow, loading all code upfront becomes inefficient. **Lazy Loading** loads feature modules on-demand, and **CanMatch** prevents modules from loading when users don't have access.

Understanding Lazy Loading Benefits

- **Reduced Initial Bundle Size:** Only load what's needed immediately
- **Faster Startup:** Application loads quicker
- **On-Demand Loading:** Features load when accessed
- **Better Performance:** Smaller bundles = faster downloads

CanMatch vs CanLoad vs CanActivate

Guard Type	When It Runs	Module Loading
CanMatch	Before route matching	Prevents module download
CanLoad	Before module loads	Prevents module download (deprecated)
CanActivate	After route matches	Module still downloads

Note:

`CanMatch` is the modern replacement for `CanLoad`. It's more flexible and integrates better with Angular's router.

Creating Admin Module

```
ng generate module admin --route admin --module app
```

Output:

```
CREATE src/app/admin/admin-routing.module.ts
```

```
CREATE src/app/admin/admin.module.ts
```

```
CREATE src/app/admin/admin.component.ts
```

```
UPDATE src/app/app-routing.module.ts
```

Note:

The `--route` flag automatically configures lazy loading for the module.

Understanding the Generated Structure

The CLI creates a separate module with its own routing configuration. Let's examine `admin-routing.module.ts`:

```
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { AdminComponent } from './admin.component';

const routes: Routes = [
  { path: '', component: AdminComponent }
];

@NgModule({
  imports: [RouterModule.forChild(routes)],
```



```
    exports: [RouterModule]
  })
  export class AdminRoutingModule { }
```

Note:

Notice `RouterModule.forChild(routes)` instead of `forRoot()`. This indicates a feature module's routing configuration.

Creating CanMatch Guard

Create `admin-match.guard.ts` in the guards folder:

```
import { inject } from '@angular/core';
import { CanMatchFn, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

/**
 * Functional guard that prevents admin module from loading
 * if user doesn't have admin privileges
 */
export const adminMatchGuard: CanMatchFn = () => {

  const auth = inject(AuthService);
  const router = inject(Router);

  console.log('CanMatch: Checking if admin module should be loaded...');

  if (auth.getRole() === 'admin') {
    console.log('CanMatch: User is admin - module will load');
    return true;
  }

  console.log('CanMatch: User is not admin - preventing module load');
  alert('Admin privileges required. Module not loaded.');
```

`router.navigate(['/']);`

```
  return false;
};
```

Note:

Functional Guards (Modern Angular): Instead of class-based guards, Angular now recommends functional guards using `CanMatchFn`. These are simpler and use the `inject()` function for dependency injection.

Configuring Lazy Loaded Route

Update `app-routing.module.ts`:

```
import { adminMatchGuard } from './guards/admin-match.guard';

const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'login', component: LoginComponent },
  {
    path: 'dashboard',
    component: DashboardComponent,
    canActivate: [AuthGuard]
  },
  {
    path: 'edit',
    component: EditProfileComponent,
    canActivate: [AuthGuard],
    canDeactivate: [DeactivateGuard]
  },
  {
    path: 'user/:id',
    component: UserDetailComponent,
    canActivate: [AuthGuard],
    resolve: { user: UserResolver }
  },
  {
    path: 'admin',
    canActivate: [adminMatchGuard], // ← Prevents loading if not admin
    loadChildren: () => import('./admin/admin.module').then(m => m.AdminModule)
  }
];
```

Note:

Dynamic Import: The `loadChildren` property uses JavaScript's dynamic `import()` syntax. This creates a separate bundle that loads only when needed.

Enhancing Admin Module

Update `admin.component.html` in the admin folder:

```
<div style="padding: 30px; background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
color: white; border-radius: 8px; margin: 20px;">
  <h2 style="margin-top: 0;">🔒 Lazy-Loaded Admin Module</h2>
  <p>This entire module was loaded on-demand. It's only downloaded when an admin user accesses
this route.</p>

  <div style="background-color: rgba(255,255,255,0.2); padding: 20px; border-radius: 4px;
margin-top: 20px;">
    <h3>Module Features:</h3>
    <ul>
      <li>☑ Loaded lazily (on-demand)</li>
      <li>☑ Protected by CanMatch guard</li>
      <li>☑ Only accessible to admins</li>
      <li>☑ Reduces initial bundle size</li>
    </ul>
  </div>

  <div style="margin-top: 20px; padding: 15px; background-color: rgba(255,255,255,0.9); color:
#333; border-radius: 4px;">
    <p style="margin: 0;"><strong>💡 Tip:</strong> Open browser DevTools > Network tab. Watch
for the admin module chunk to load when you navigate here!</p>
  </div>
</div>
```

Output:**Testing Lazy Loading:**

1. Open browser DevTools > Network tab
2. Clear network log

3. Login as regular user
4. Try to access `/admin`
5. Result: No admin module file appears in Network tab (not downloaded)
6. Logout and login as admin
7. Navigate to `/admin`
8. Result: A new JavaScript chunk file appears in Network tab
9. File name will be something like: `admin-admin-module.js`

✓ **Success:**

Achievement Unlocked: Admin module loads on-demand and only for authorized users, optimizing application performance.

STEP 9: Combined Protection System

Real-world applications often require multiple layers of protection on a single route. Let's create a comprehensive example that combines multiple guards, resolvers, and data.

Creating Product Components and Service

```
ng g c pages/product-edit
ng g s services/product
```

Implementing Product Service

Open `product.service.ts`:

```
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { delay } from 'rxjs/operators';
```

```
export interface Product {
  id: number;
  name: string;
  price: number;
  category: string;
  stock: number;
}

@Injectable({
  providedIn: 'root'
})
export class ProductService {

  private products: Product[] = [
    { id: 1, name: 'Laptop Pro', price: 1299, category: 'Electronics', stock: 45 },
    { id: 2, name: 'Wireless Mouse', price: 29, category: 'Accessories', stock: 150 },
    { id: 3, name: 'USB-C Hub', price: 49, category: 'Accessories', stock: 200 }
  ];

  getProduct(id: number): Observable<Product | undefined> {
    const product = this.products.find(p => p.id === id);
    return of(product).pipe(delay(800));
  }

  updateProduct(product: Product): Observable<boolean> {
    // Simulate API update
    return of(true).pipe(delay(500));
  }
}
```

Creating Product Resolver

```
ng g resolver resolvers/product
```

Open `product.resolver.ts` :

```
import { Injectable } from '@angular/core';
import { Resolve, ActivatedRouteSnapshot } from '@angular/router';
import { Observable } from 'rxjs';
import { ProductService, Product } from '../services/product.service';
```

```
@Injectable({
  providedIn: 'root'
})
export class ProductResolver implements Resolve<Product | undefined> {

  constructor(private productService: ProductService) {}

  resolve(route: ActivatedRouteSnapshot): Observable<Product | undefined> {
    const productId = +route.paramMap.get('id')!;
    console.log(`Product Resolver: Loading product ${productId}...`);
    return this.productService.getProduct(productId);
  }
}
```

Implementing Product Edit Component

Update `product-edit.component.ts`:

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { Product, ProductService } from '../../services/product.service';
import { CanLeave } from '../../guards/can-leave.interface';

@Component({
  selector: 'app-product-edit',
  templateUrl: './product-edit.component.html',
  styleUrls: ['./product-edit.component.css']
})
export class ProductEditComponent implements OnInit, CanLeave {

  product: Product | undefined;
  originalProduct: Product | undefined;
  dirty = false;

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private productService: ProductService
  ) {}

  ngOnInit(): void {
    // Get pre-loaded product from resolver
    this.product = this.route.snapshot.data['product'];
    this.originalProduct = { ...this.product! };
  }
}
```

```

    console.log('Product Edit Component initialized with:', this.product);
  }

  onFieldChange(): void {
    this.dirty = true;
  }

  save(): void {
    if (this.product) {
      this.productService.updateProduct(this.product).subscribe(() => {
        this.dirty = false;
        alert('Product updated successfully!');
        this.router.navigate(['/dashboard']);
      });
    }
  }

  canDeactivate(): boolean {
    if (this.dirty) {
      return confirm('You have unsaved changes to this product. Leave anyway?');
    }
    return true;
  }
}

```

Update `product-edit.component.html` :

```

<div style="max-width: 700px; margin: 30px auto; padding: 30px; border: 1px solid #ddd; border-
radius: 8px; background-color: #fff;">

  <div style="background: linear-gradient(135deg, #667eea 0%, #764ba2 100%); color: white;
padding: 20px; border-radius: 8px; margin-bottom: 30px;">
    <h2 style="margin: 0;"><img alt="edit icon" data-bbox="330 655 345 670"/> Edit Product</h2>
    <p style="margin: 5px 0 0 0; opacity: 0.9;">Modify product details (Admin Only)</p>
  </div>

  <div *ngIf="product">
    <div style="margin-bottom: 20px;">
      <label style="display: block; margin-bottom: 5px; font-weight: 600;">Product ID:</label>
      <input
        type="text"
        [value]="product.id"
        readonly
        style="width: 100%; padding: 10px; box-sizing: border-box; background-color: #f0f0f0;
border: 1px solid #ddd; border-radius: 4px;">
    </div>
  </div>

```

```

</div>

<div style="margin-bottom: 20px;">
  <label style="display: block; margin-bottom: 5px; font-weight: 600;">Product Name:</label>
  <input
    type="text"
    [(ngModel)]="product.name"
    (input)="onFieldChange()"
    style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;">
</div>

<div style="margin-bottom: 20px;">
  <label style="display: block; margin-bottom: 5px; font-weight: 600;">Price ($):</label>
  <input
    type="number"
    [(ngModel)]="product.price"
    (input)="onFieldChange()"
    style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;">
</div>

<div style="margin-bottom: 20px;">
  <label style="display: block; margin-bottom: 5px; font-weight: 600;">Category:</label>
  <input
    type="text"
    [(ngModel)]="product.category"
    (input)="onFieldChange()"
    style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;">
</div>

<div style="margin-bottom: 20px;">
  <label style="display: block; margin-bottom: 5px; font-weight: 600;">Stock:</label>
  <input
    type="number"
    [(ngModel)]="product.stock"
    (input)="onFieldChange()"
    style="width: 100%; padding: 10px; box-sizing: border-box; border: 1px solid #ddd;
border-radius: 4px;">
</div>

<button
  (click)="save()"
  style="padding: 12px 30px; background-color: #28a745; color: white; border: none; border-
radius: 4px; cursor: pointer; margin-right: 10px; font-size: 1em;">

```



```

     Save Changes
  </button>

  <button
    routerLink="/dashboard"
    style="padding: 12px 30px; background-color: #6c757d; color: white; border: none; border-
radius: 4px; cursor: pointer; font-size: 1em;">
    Cancel
  </button>

  <div *ngIf="dirty" style="margin-top: 20px; padding: 15px; background-color: #fff3cd;
border-left: 4px solid #ffc107; border-radius: 4px;">
    <strong>⚠ Unsaved Changes</strong>
    <p style="margin: 5px 0 0 0;">You have modified this product. Remember to save before
leaving!</p>
  </div>
</div>

<div *ngIf="!product" style="text-align: center; padding: 30px; color: #dc3545;">
  <h3>✖ Product Not Found</h3>
  <p>The requested product does not exist.</p>
</div>

</div>

```

Configuring the Complete Route

Update `app-routing.module.ts` with the comprehensive product edit route:

```

import { ProductResolver } from './resolvers/product.resolver';
import { ProductEditComponent } from './pages/product-edit/product-edit.component';

const routes: Routes = [
  // ... existing routes ...

  {
    path: 'products/:id/edit',
    component: ProductEditComponent,

    // Multiple guards execute in order
    canActivate: [AuthGuard, RoleGuard],

    // Warn before leaving with unsaved changes
    canDeactivate: [DeactivateGuard],
  }
];

```

```
// Pre-load product data
resolve: { product: ProductResolver },

// Pass required role to guard
data: { role: 'admin' }
}
];
```

Adding Product Links to Dashboard

Update `dashboard.component.html` to include product edit links:

```
<div style="padding: 30px;">
  <h2><img alt="Dashboard icon" data-bbox="158 388 178 403"/> Dashboard</h2>
  <p>Welcome to your personal dashboard. This area is protected and requires authentication.</p>

  <!-- User Directory Section -->
  <div style="margin-top: 30px; background-color: #f8f9fa; padding: 20px; border-radius: 8px;">
    <h3>User Directory</h3>
    <p>Click on a user to view their profile (demonstrates resolver functionality):</p>

    <ul style="list-style: none; padding: 0;">
      <li style="margin: 10px 0;">
        <a routerLink="/user/1" style="display: block; padding: 15px; background-color: white;
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
          <img alt="User icon" data-bbox="191 588 206 603"/> View John Doe's Profile
        </a>
      </li>
      <li style="margin: 10px 0;">
        <a routerLink="/user/2" style="display: block; padding: 15px; background-color: white;
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
          <img alt="User icon" data-bbox="191 681 206 696"/> View Jane Smith's Profile
        </a>
      </li>
      <li style="margin: 10px 0;">
        <a routerLink="/user/3" style="display: block; padding: 15px; background-color: white;
border: 1px solid #ddd; border-radius: 4px; text-decoration: none; color: #333;">
          <img alt="User icon" data-bbox="191 774 206 789"/> View Bob Johnson's Profile
        </a>
      </li>
    </ul>
  </div>
```

```

<!-- Product Management Section (Admin Only) -->
<div *ngIf="auth.getRole() === 'admin'" style="margin-top: 30px; background-color: #e7f3ff;
padding: 20px; border-radius: 8px; border-left: 4px solid #007bff;">
  <h3><img alt="Product icon" data-bbox="175 195 195 210"/> Product Management (Admin Only)</h3>
  <p>Edit products - demonstrates combined guards and resolver:</p>

  <ul style="list-style: none; padding: 0;">
    <li style="margin: 10px 0;">
      <a routerLink="/products/1/edit" style="display: block; padding: 15px; background-
color: white; border: 1px solid #007bff; border-radius: 4px; text-decoration: none; color:
#333;">
        <img alt="Edit icon" data-bbox="185 315 205 330"/> Edit Laptop Pro
      </a>
    </li>
    <li style="margin: 10px 0;">
      <a routerLink="/products/2/edit" style="display: block; padding: 15px; background-
color: white; border: 1px solid #007bff; border-radius: 4px; text-decoration: none; color:
#333;">
        <img alt="Edit icon" data-bbox="185 425 205 440"/> Edit Wireless Mouse
      </a>
    </li>
    <li style="margin: 10px 0;">
      <a routerLink="/products/3/edit" style="display: block; padding: 15px; background-
color: white; border: 1px solid #007bff; border-radius: 4px; text-decoration: none; color:
#333;">
        <img alt="Edit icon" data-bbox="185 530 205 545"/> Edit USB-C Hub
      </a>
    </li>
  </ul>

  <div style="margin-top: 15px; padding: 15px; background-color: rgba(255,255,255,0.7);
border-radius: 4px;">
    <p style="margin: 0; font-size: 0.9em;"><strong><img alt="Shield icon" data-bbox="555 640 575 655"/> Protection Layers:</strong></p>
    <ul style="margin: 5px 0; font-size: 0.9em;">
      <li><input checked="" type="checkbox"/> Authentication required (AuthGuard)</li>
      <li><input checked="" type="checkbox"/> Admin role required (RoleGuard)</li>
      <li><input checked="" type="checkbox"/> Data pre-loaded (ProductResolver)</li>
      <li><input checked="" type="checkbox"/> Unsaved changes warning (DeactivateGuard)</li>
    </ul>
  </div>
</div>
</div>

```

Update `dashboard.component.ts` to make auth service public:

```
import { Component } from '@angular/core';
import { AuthService } from '../../services/auth.service';

@Component({
  selector: 'app-dashboard',
  templateUrl: './dashboard.component.html',
  styleUrls: ['./dashboard.component.css']
})
export class DashboardComponent {

  constructor(public auth: AuthService) {}
}
```

Output:

Testing the Complete System:

1. Login as regular user
2. Dashboard shows user profiles but no product section
3. Try accessing `/products/1/edit` directly
4. Result: Blocked by RoleGuard
5. Logout and login as admin
6. Product Management section now visible
7. Click "Edit Laptop Pro"
8. Observe delay as ProductResolver loads data
9. Modify any field
10. Try to navigate away
11. Result: Confirmation dialog appears
12. Click "Save Changes"
13. Now navigate away smoothly

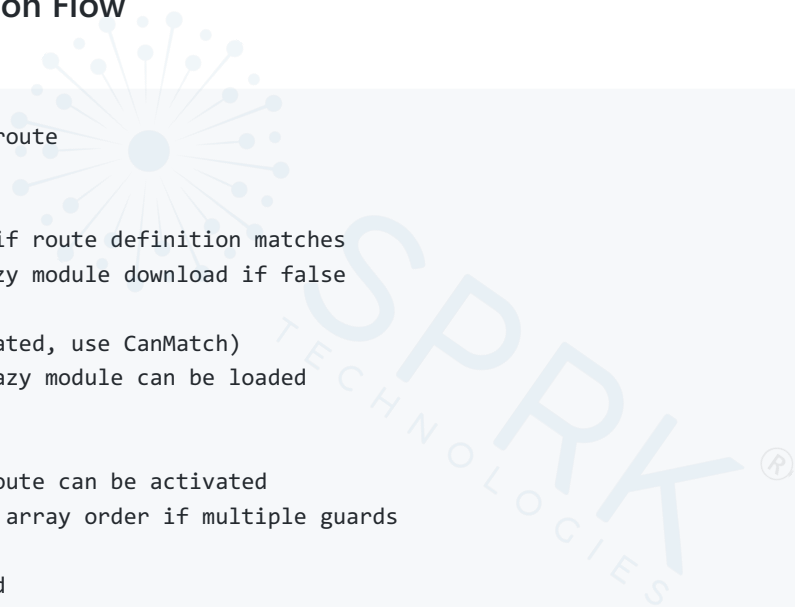
✓ Success:

Achievement Unlocked: Complete multi-layer protection system is operational with authentication, authorization, data pre-loading, and unsaved changes protection working together seamlessly.

Route Guard Execution Order

Understanding the sequence in which Angular executes guards and resolvers is crucial for building reliable applications.

Complete Execution Flow



```
graph TD
    A[User navigates to route] --> B[1. CanMatch]
    B --> B1[Determines if route definition matches]
    B --> B2[Prevents lazy module download if false]
    B1 --> C[2. CanLoad deprecated, use CanMatch]
    B2 --> C
    C --> C1[Checks if lazy module can be loaded]
    C1 --> D[3. CanActivate]
    D --> D1[Checks if route can be activated]
    D --> D2[Executes in array order if multiple guards]
    D1 --> E[4. CanActivateChild]
    D2 --> E
    E --> E1[Checks if child routes can be activated]
    E1 --> F[5. Resolve]
    F --> F1[Fetches required data]
    F --> F2[Navigation waits for completion]
    F1 --> G[6. Component Initialized]
    F2 --> G
    G --> G1[ngOnInit runs]
    G --> G2[Component displays]
    G1 --> H["(User interacts, then tries to leave)"]
    G2 --> H
    H --> I[ ]
```

User navigates to route

↓

1. CanMatch
 - ↳ Determines if route definition matches
 - ↳ Prevents lazy module download if false
2. CanLoad (deprecated, use CanMatch)
 - ↳ Checks if lazy module can be loaded
3. CanActivate
 - ↳ Checks if route can be activated
 - ↳ Executes in array order if multiple guards
4. CanActivateChild
 - ↳ Checks if child routes can be activated
5. Resolve
 - ↳ Fetches required data
 - ↳ Navigation waits for completion
6. Component Initialized
 - ↳ ngOnInit runs
 - ↳ Component displays

↓

(User interacts, then tries to leave)

↓

7. CanDeactivate

- ↳ Checks if user can leave component
- ↳ Runs when navigating away

Note:

Key Points:

- All guards must return `true` for navigation to proceed
- If any guard returns `false`, navigation is cancelled
- Guards can be synchronous (boolean) or asynchronous (Promise/Observable)
- Resolvers delay navigation until data is available

Project Structure Summary

Our completed application has the following organized structure:

```
src/app/
├── guards/
│   ├── auth.guard.ts           // Authentication guard
│   ├── role.guard.ts          // Role-based authorization
│   ├── deactivate.guard.ts     // Unsaved changes protection
│   ├── admin-child.guard.ts    // Child route protection
│   ├── admin-match.guard.ts    // Lazy loading permission
│   └── can-leave.interface.ts  // Deactivation contract
├── services/
│   ├── auth.service.ts        // Authentication logic
│   ├── user.service.ts        // User data operations
│   └── product.service.ts      // Product data operations
├── resolvers/
│   ├── user.resolver.ts        // User data pre-loading
│   └── product.resolver.ts     // Product data pre-loading
└── pages/
    └── home/
```

```
| | login/
| | dashboard/
| | edit-profile/
| | admin/
| | user-detail/
| | └─ product-edit/
|
| └─ admin/                                // Lazy-loaded module
|   | admin.module.ts
|   | admin-routing.module.ts
|   | admin.component.ts
|   | users/
|   | └─ settings/
|
| └─ app-routing.module.ts                // Main routing configuration
| └─ app.module.ts
| └─ app.component.ts
```

Best Practices and Recommendations

1. Guard Design Principles

- **Single Responsibility:** Each guard should have one clear purpose
- **Reusability:** Design guards to be reusable across multiple routes
- **User Communication:** Provide clear feedback when access is denied
- **Performance:** Keep guard logic lightweight and fast

2. Security Considerations

⚠ Important:

Client-Side Guards Are Not Enough: Route guards only prevent navigation in the browser. They don't secure your backend API. Always implement server-side authentication and authorization.

- Never store sensitive information in client-side code
- Always validate user permissions on the server

- Use HTTPS in production
- Implement proper token management (JWT, OAuth)
- Guards are for UX, not security

3. Performance Optimization

- Use lazy loading for feature modules
- Keep resolver data fetching efficient
- Consider caching strategies for frequently accessed data
- Use CanMatch instead of deprecated CanLoad

4. User Experience

- Provide loading indicators during resolver operations
- Give clear error messages when guards block navigation
- Implement proper redirect flows after authentication
- Save form state to prevent data loss

Common Patterns and Use Cases

Pattern 1: Role-Based Access Control

```
// Routes with different permission levels
{
  path: 'dashboard',
  canActivate: [AuthGuard] // Any logged-in user
},
{
  path: 'admin',
  canActivate: [AuthGuard, RoleGuard], // Only admins
  data: { role: 'admin' }
}
```


Pattern 2: Feature Module Protection

```
// Lazy-loaded module with access control
{
  path: 'premium',
  canMatch: [premiumUserGuard],
  loadChildren: () => import('./premium/premium.module')
    .then(m => m.PremiumModule)
}
```

Pattern 3: Data Pre-loading

```
// Ensure data is ready before component renders
{
  path: 'details/:id',
  component: DetailsComponent,
  resolve: {
    data: DataResolver,
    config: ConfigResolver
  }
}
```

Pattern 4: Complex Forms

```
// Protect against accidental navigation from forms
{
  path: 'edit/:id',
  component: EditFormComponent,
  canActivate: [AuthGuard],
  canDeactivate: [UnsavedChangesGuard],
  resolve: { formData: FormDataResolver }
}
```

Debugging and Troubleshooting

Common Issues and Solutions

Issue	Solution
Guard not executing	Verify guard is correctly imported and added to route configuration
Infinite redirect loops	Ensure guards don't redirect to routes that trigger the same guard
Resolver data not available	Check that resolver key in route config matches component access
CanDeactivate not working	Verify component implements the required interface
Lazy module still downloads	Use CanMatch instead of CanActivate for lazy routes

Debugging Techniques

```
// Add console logs to track guard execution
canActivate(): boolean {
  console.log('AuthGuard: Checking authentication...');
  const isAuthenticated = this.auth.isLoggedIn();
  console.log('AuthGuard: User authenticated:', isAuthenticated);
  return isAuthenticated;
}
```

Learning Path Recommendation

To master route protection, practice in this order:

1. **Week 1:** Basic routing and navigation

2. **Week 2:** Authentication service and CanActivate
 3. **Week 3:** CanDeactivate for form protection
 4. **Week 4:** Role-based guards and authorization
 5. **Week 5:** Child route protection with CanActivateChild
 6. **Week 6:** Resolvers for data pre-loading
 7. **Week 7:** Lazy loading with CanMatch
 8. **Week 8:** Build a complete project combining all concepts
-

Conclusion

Route protection is a fundamental aspect of building secure and user-friendly Angular applications. Through this progressive project, we've explored:

- ☒ **CanActivate:** Controlling access to routes
- ☒ **CanDeactivate:** Preventing accidental data loss
- ☒ **CanActivateChild:** Efficiently protecting nested routes
- ☒ **CanMatch:** Optimizing lazy-loaded modules
- ☒ **Resolvers:** Ensuring data availability
- ☒ **Combined Systems:** Layering multiple protection mechanisms

☒ **Success:**

Key Takeaway: Route guards are essential for both security and user experience. They work together to create robust, professional Angular applications that protect sensitive areas while maintaining a smooth user journey.

Important:

Remember: Client-side route guards are only part of the security picture. Always implement proper backend authentication and authorization to truly secure your application.

Next Steps

Continue your Angular journey by exploring:

- HTTP Interceptors for token management
- State management with NgRx or Akita
- Real authentication with JWT tokens
- Advanced RxJS for complex async operations
- Testing strategies for guards and resolvers

Happy coding! 🚀

